1. Implement a support vector machine to separate and classify the two linearly separable sets given in the data file SVM_data.mat available on Canvas.

Find and show the classifying line. How many support vectors do you have? Mark your support vectors.

Solution:

```
import scipy.io
import numpy as np
from sklearn.svm import SVC
import matplotlib.pyplot as plt
import warnings



# Loadimg the data from the mat files
data = scipy.io.loadmat('/content/drive/MyDrive/My/Machine learning Files/SVM_data.mat')
x = data['x']
y = data['y'].ravel()

# Fitting a linear SVM model
model = SVC(kernel='linear')
model.fit(x, y)

# Getting the decision boundary lines parameters
w = model.coef_[0]
b = model.intercept_[0]
slope = -w[0] / w[1]
intercept = -b / w[1]
margin = 1 / np.sqrt(np.sum(model.coef_ ** 2))

# Calculating the decision boundary line and margins for the plot
xx = np.linspace(x[:, 0].min() - 1, x[:, 0].max() + 1, 100)
yy = slope * xx + intercept
yy_down = yy - np.sqrt(1 + slope ** 2) * margin
yy_up = yy + np.sqrt(1 + slope ** 2) * margin

x_ticks = np.arange(-4, 5, 1)
y_ticks = np.arange(-10, 9, 2)

#generating the plot
plt.figure(figsize=(10,10))

# Plotting of the dataset
plt.scatter(x[:, 0], x[:, 1], c=y, cmap='bwr', s=50)

# Plotting the decision boundary line
plt.plot(xx, yy, 'k-')
```

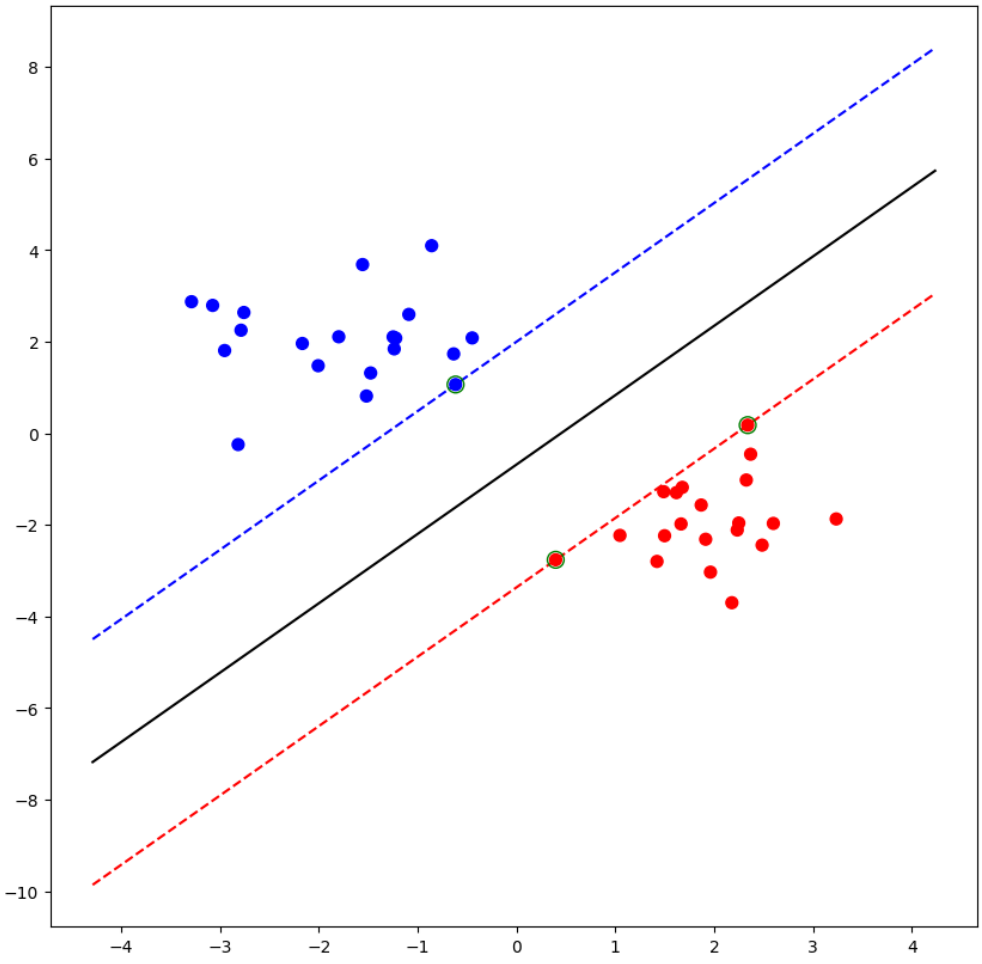
```
plt.plot(xx, yy_down, 'r--')
plt.plot(xx, yy_up, 'b--')

# support vectors are being highlighted by green color
plt.scatter(model.support_vectors_[:, 0], model.support_vectors_[:, 1],
            s=100, facecolors='none', edgecolors='green', zorder=10)

plt.xticks(x_ticks)
plt.yticks(y_ticks)


plt.show()
```

Output:



2. Implement a kernel-based support vector machine to separate and classify the two nonlinearly separable sets given in the data file SVM_data_nonlinear.mat available on Canvas.

Find and show the classifying line. How many support vectors do you have? Mark your support vectors.

```
import scipy.io
import numpy as np
import matplotlib.pyplot as plt
from sklearn.svm import SVC

# Load the data from SVM_data_nonlinear.mat
data_nonlinear = scipy.io.loadmat('/content/drive/MyDrive/My/Machine
learning Files/SVM_data_nonlinear.mat')

# Extracting x (features) and y (labels)
x_nonlinear = data_nonlinear['x']
y_nonlinear = data_nonlinear['y'].flatten()

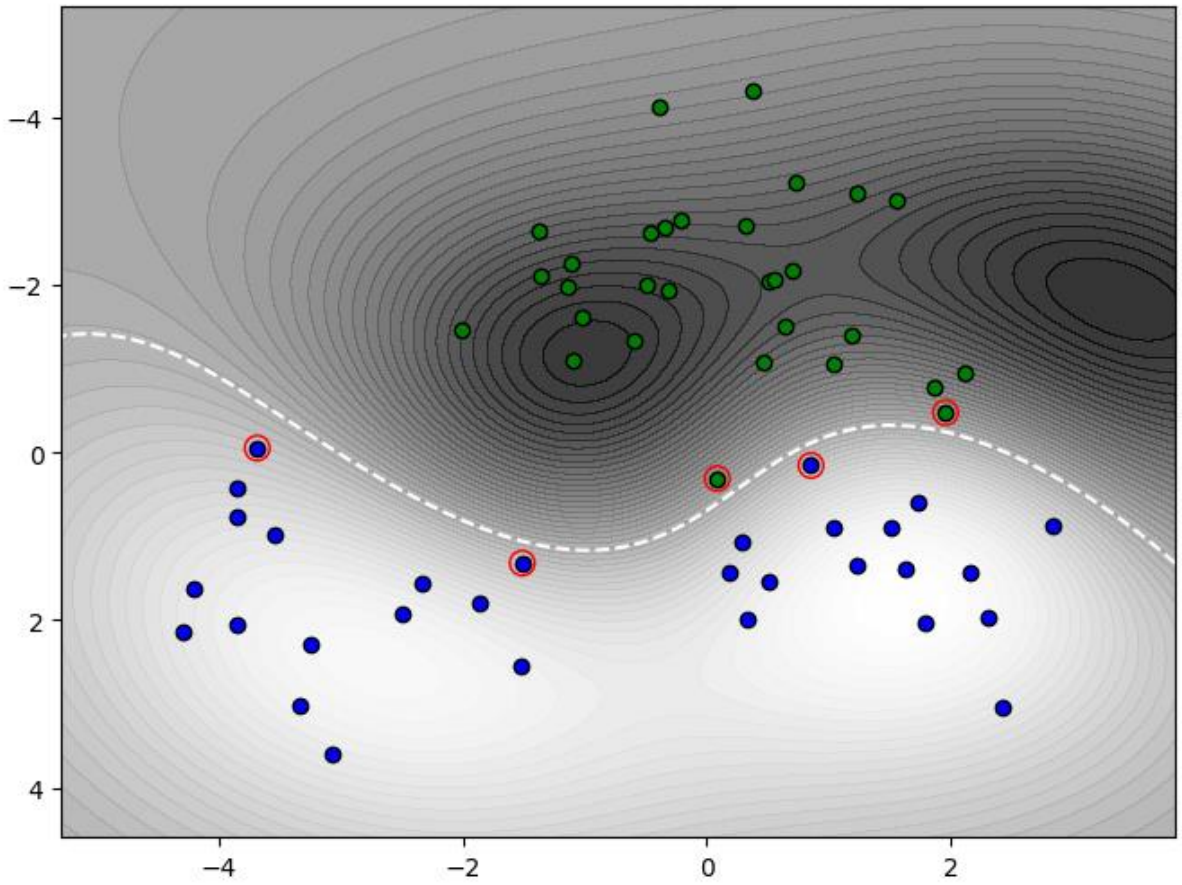
# Implementing kernel-based SVM for the non-linearly separable data
svm_nonlinear = SVC(kernel='rbf', C=1E10)
svm_nonlinear.fit(x_nonlinear, y_nonlinear)

# Preparing the meshgrid for decision boundary visualization
xx, yy = np.meshgrid(np.linspace(x_nonlinear[:, 0].min() - 1,
x_nonlinear[:, 0].max() + 1, 500),
                     np.linspace(x_nonlinear[:, 1].min() - 1,
x_nonlinear[:, 1].max() + 1, 500))
Z = svm_nonlinear.decision_function(np.c_[xx.ravel(), yy.ravel()])
Z = Z.reshape(xx.shape)

# Adjusting the visualization with the specified color and style
changes
plt.figure(figsize=(8, 6))
plt.contourf(xx, yy, Z, levels=np.linspace(Z.min(), Z.max(), 50),
cmap='Greys', alpha=0.8)
plt.contour(xx, yy, Z, colors='white', levels=[0], linestyles='--')
plt.scatter(x_nonlinear[y_nonlinear == 1][:, 0],
x_nonlinear[y_nonlinear == 1][:, 1], c='green', edgecolors='k',
label='Class 1')
plt.scatter(x_nonlinear[y_nonlinear == -1][:, 0],
x_nonlinear[y_nonlinear == -1][:, 1], c='blue', edgecolors='k',
label='Class -1')
plt.scatter(svm_nonlinear.support_vectors_[:, 0],
svm_nonlinear.support_vectors_[:, 1], s=100, linewidth=1,
facecolors='none', edgecolors='red')

plt.gca().invert_yaxis()
plt.show()
```

Output:



- From the above Output I have selected 5 support vectors and are marked with red spots.